

DNS Resolver using AF_XDP*

Aarsh WANKAR (23110003)^{1,4} Abhinav KHOT (23110006)^{1,4}
 Jaskirat SINGH Maskeen (23110146),^{1,4} Karan SAGAR Gandhi (23110157),^{1,4}
 Naveen TIWARI^{1,2} and Sameer G. Kulkarni^{1,3}

¹Student, Computer Science and Engineering Department, Indian Institute of Technology, Gandhinagar

²Teaching assistant, CS 331 Computer Networks

³Assistant Professor, Computer Science and Engineering Department

⁴Equal Contributions, Ordering based on roll numbers

This project report presents a working DNS resolver that uses the XDP sockets for processing DNS packets which completely bypass the linux kernel networking stack and directly filter and process packets into a resolver application in the user space. We demonstrate that the technique leads to better latencies in terms of query processing and also demonstrate the effects on CPU overhead in terms of cycles taken to resolve each query.

Key Words: DNS, AF_XDP, eBPF

GitHub Repository for the project

1. Introduction

Most high-speed network applications have strict requirements on speed of DNS resolution but the Linux networking stack applies significant overhead on packet processing after the packet has been received as it has to travel through the kernel to be copied into the user space. The main idea behind using XDP sockets is to bypass this overhead and copy these packets directly from the NIC (Network Interface Card) to the user space.

To understand the working of our implementation, we first explore how a packet traditionally passes through the linux network stack and what overheads we are trying to reduce in these paths.

1.1. Journey of a packet through the network stack

The flow of packets happens in two directions, ingress and egress. We explore these paths as described by:⁵⁾

- **Ingress** : The packet is first recieved at the NIC (Network Interface card). The NIC uses Recieve and Transmission (Rx and Tx) Rings to communicate with the kernel. Once a packet has been received, the NIC copies it to the system memory using **Direct Memory Access (DMA)**. After this, it notifies the operating system by raising an interrupt and indicates the location of the packet. The OS then allocates a data structure called the **Socket Buffer or sk_buff** to store metadata of the packet and the start and end pointers of the packet in memory. An usual structure is given in Figure 1. The packet then travels up to the IP layer where it is processed by functions such as

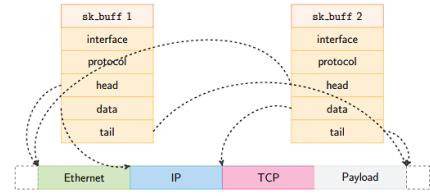


Fig. 1. Structure of an Socket buffer (sk_buf)

`ip_rcv()`, `ip_forward()` et cetera. The packet is then processed by the Transport layer using functions like `tcp_v4_rcv()`, `__skb_rcv_udp()` et cetera before finally reaching the application layer. The above indicates immense amount of processing before the packet even reaches the application that requires it.

- **Egress**: The egress path starts when a user-space application uses a system call like `write()`, which invokes `sock_sendmsg()` in the kernel. This calls the appropriate transport layer function, such as `tcp_sendmsg()` or `udp_sendmsg()`, which allocates an `sk_buff`, builds the transport header, and enqueues the data in the socket write queue. The packet is then passed to the IP layer via `_ip_queue_xmit()`. This function determines the route, builds the IP header, handles fragmentation, and uses ARP (Address Resolution Protocol) to find the destination MAC address. Finally, the kernel constructs the Ethernet header and calls `dev_queue_xmit()`, which queues the packet in the queueing discipline (qdisc) before it's added to the NIC's TX ring. The packet is then mapped for **Direct Memory Access (DMA)**, and the NIC is notified via an interrupt to transmit it.

A complete overview of the main functions involved in the above two processes, for UDP, are illustrated in Figure 2.

Within the kernel, the `sk_buff` datastructure is used and cloned very efficiently. A clone only copies the pointers to the data and not the data itself. The main overhead posed by `sk_buff` is due to a lot of these structures being maintained. These structures are very large in size (~ 320 bytes) and of-

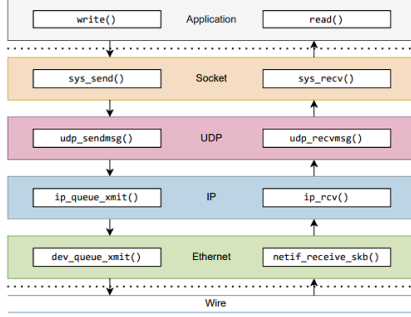


Fig. 2. Important functions involved in egress and ingress in the linux kernel stack for UDP

ten lead to expensive cache misses.²⁾ But the main problem happens to be the copying of the payload from the NIC to the kernel memory and the copying of the application layer payload from the kernel to the user space. These are unavoidable copies and we cannot just use pointers to data blocks as that would violate the separation of user-space and kernel-space.

1.2. eBPF (Extended Berkley Packet Filter)

eBPF is a sandboxed virtual machine through which we are able to run custom user defined programs at various points in the kernel (which are also known as **hooks**). It is extremely useful to implement custom packet processing logic in the predefined linux kernel. In this project, we use the XDP hook to run an application that filters DNS packets and then redirects them to the user-space application. Details about the exact working of this are deferred to later sections. The main idea is that, we process the incoming packet and check if it is an IPv4 packet and if its destination port is 53 and if it is a UDP packet and make the following decisions:

- **XDP_PASS** : Not a DNS query, send it through the regular kernel path.
- **XDP_DROP** : Drop this packet immediately
- **XDP_REDIRECT** : DNS query packet detected, send this directly to user space using the XDP mechanism.

1.3. XDP (Express Data Path)

When a packet arrives at the NIC, the NIC computes an RSS (Receive Side Scaling) hash-typically over a 2–5 tuple (commonly 5-tuple for UDP/TCP, depending on NIC configuration)-to select an RX queue. Note that in our implementation, we only have one RX queue for the NIC. Each RX queue is a hardware ring buffer of *DMA descriptors*, where each descriptor contains a pre-programmed DMA address pointing to a packet buffer. In zero-copy AF_XDP mode, these DMA buffer addresses belong to a region of user memory called **UMEM**, which has been pre-registered and pinned by the driver.

The NIC writes the packet into the next available descriptor's associated DMA address using DMA. During this step, the CPU is not involved. In modern high-performance drivers, RX processing commonly occurs in **NAPI poll mode** or **AF_XDP busy-poll mode**, where packet processing does not trigger an interrupt per packet.

Once a batch of packets is ready, the driver invokes the XDP hook via a function pointer previously installed when

the XDP program was `attacheueched` (rather than dynamically checking for NULL). The XDP program receives packet boundaries through the `xdp_md` metadata struct, which provides offsets into the driver-owned buffer-backed by UMEM when zero-copy is enabled.

In the DNS server case, if the XDP hook program (which we attach) identifies a packet destined for port 53, it calls:

```
bpf_redirect_map(&xsks_map, rx_queue_index, 0);
```

The **XSKMAP** holds kernel-space references to AF_XDP socket objects (`struct xdp_sock *`), indexed by NIC RX queue ID. The XDP program does **not** itself deliver the packet, copy memory, or interact with the socket directly. It only selects a target socket and returns **XDP_REDIRECT**.

After the XDP program returns, the kernel performs the redirect. It takes the same UMEM buffer address that the NIC used for DMA and writes it into the selected socket's **RX ring**. Importantly:

- The packet is **not copied**.
- Only the UMEM buffer address is enqueued in a lock-free ring.
- The NIC plays no role after DMA completes.

Userspace continuously polls the AF_XDP socket. When a new buffer address appears in the socket's RX ring, userspace directly accesses and processes the packet from UMEM.

In contrast, traditional kernel sockets do not support zero-copy in the general case. Packets are first DMA-written into kernel memory, wrapped in multiple `sk_buff` structures as they traverse the network stack, and finally copied into userspace before being made available to the application.

Thus, AF_XDP in zero-copy mode eliminates:

1. Allocation and traversal of `sk_buff` structures across network stack layers,
2. Per-packet system calls and kernel/userspace transitions,
3. And the final memory copy from kernel space to user space.

Instead, AF_XDP delivers packets by publishing UMEM buffer addresses through a shared ring, enabling high-throughput, low-latency packet processing for workloads such as DNS servers.

1.4. AF_XDP Egress Path

When the server has generated a response (e.g., a DNS reply), it transmits the packet using the AF_XDP zero-copy pipeline. The buffer used to hold the incoming request resides in the shared UMEM region and is already DMA mapped, allowing it to be directly reused for transmission without any additional memory copies.

To send a packet, the userspace process writes the response payload into a UMEM frame and inserts the frame address into the AF_XDP **TX ring**, indicating that the buffer is ready for transmission. A trigger such as a zero-length `sendto()` call is issued to notify the kernel to process the TX ring. The kernel driver then dequeues these UMEM addresses and writes them into the NIC's **TX descriptor ring**, configuring the NIC with the DMA source address and packet length.

The NIC performs a DMA read from the UMEM region into its internal buffers and transmits the packet on the wire,

without CPU involvement in the data movement. Once transmission completes, the NIC signals completion via an interrupt or NAPI poll, upon which the driver places the corresponding UMEM frame address into the **AF_XDP Completion ring**. Userspace polls this ring to learn which buffers have finished transmission and recycles them by pushing their addresses back into the **FILL ring**, making them available for future packet reception.

Unlike the traditional network stack, this path involves no creation of `sk_buff` structures, no traversal of the kernel networking layers, and no user-to-kernel or kernel-to-NIC data copies. By allowing the same DMA-mapped UMEM buffer to be reused across RX, processing, and TX, AF_XDP eliminates memory copies and minimizes kernel involvement, significantly reducing per-packet overhead while maintaining direct control of buffer lifetime in userspace.

1.5. AF_XDP Socket Architecture

An AF_XDP socket operates on a shared memory region called *UMEM* (User Memory), which holds packet buffers accessible to both userspace and the NIC through DMA. Rather than copying packets between kernel and user memory, AF_XDP exchanges UMEM buffer references using a set of lockless rings, each denoting ownership of packet buffers at a given stage.

The **FILL ring** is populated by userspace with UMEM frame offsets representing empty buffers available for packet reception. The kernel driver consumes these entries to program the NIC's RX descriptors with DMA addresses, allowing the NIC to write incoming packet data directly into user-allocated memory. Once the NIC has written a packet, and if redirected to an AF_XDP socket via XDP, the same buffer reference is placed into the **RX ring**, which userspace polls to consume received packets.

For packet transmission, userspace prepares a response directly inside a UMEM buffer and enqueues its reference into the **TX ring**. The driver dequeues entries from the TX ring and programs the NIC's TX descriptors, enabling the NIC to DMA-read packet data and send it on the network. After transmission completes, the driver places the buffer reference in the **COMPLETION ring**, signalling to userspace that the buffer is no longer in use by the NIC and may be recycled. Userspace typically returns these buffers back to the **FILL ring** for reuse in future reception events.

Throughout this process, the rings do not store packet contents or kernel metadata, but only UMEM buffer offsets, enabling a zero-copy data path. By delegating buffer lifecycle management to userspace while offloading data movement to DMA, AF_XDP removes the need for `sk_buff` allocations and avoids traversing the kernel networking stack, significantly reducing per-packet overhead while maintaining full control of packet processing in userspace.

2. Our Setup

To ensure a consistent, reproducible, and isolated environment for building and testing our DNS resolver, we utilize a Docker container. The entire environment, including all dependencies, is defined in a `Dockerfile`.

2.1. Overview of Packages Installed

Our setup is based on an `ubuntu:22.04` container image. Upon this base, we install packages required for building the eBPF programs, the user-space applications, and the necessary testing utilities. Some of them are:

- **Compilers and Build Tools:** `build-essential`, `clang`, `llvm`, `pkg-config`, and `m4`. `clang` and `llvm` are specifically required for compiling the eBPF kernel program.
- **eBPF/XDP Libraries:** `libelf-dev` and `libpcap-dev` are installed to support eBPF object file loading and packet handling.
- **Benchmarking and Networking Tools:** We install `iproute2`, `net-tools`, `iputils-ping`, and `linux-tools-generic` to provide utilities for network configuration, testing, and performance analysis with the `perf` tool.

2.2. The Build Process

The build process for the resolver and its dependencies is automated within the container. The build proceeds in several stages:

1. The source code for the project is copied into the `/src` directory in the container.
2. The main project is configured and built using `./configure` and `make`, respectively.
3. The `xdp-tools` library, is built from the source included in our project tree (`/src/lib/xdp-tools`).
4. For the non-XDP comparison, the standard server executables (`server` and `server.cached`) are compiled directly using `gcc`.

2.3. Running the XDP Resolver

To run the AF-XDP version of our resolver, the Docker container must be launched with specific flags to grant it the necessary permissions for low-level network operations. The container requires the `--privileged` flag to attach the XDP program to the network interface. Furthermore, the memory lock limit must be raised (`--ulimit memlock=-1`), which is a requirement for creating and using the shared UMEM (User Memory) region.

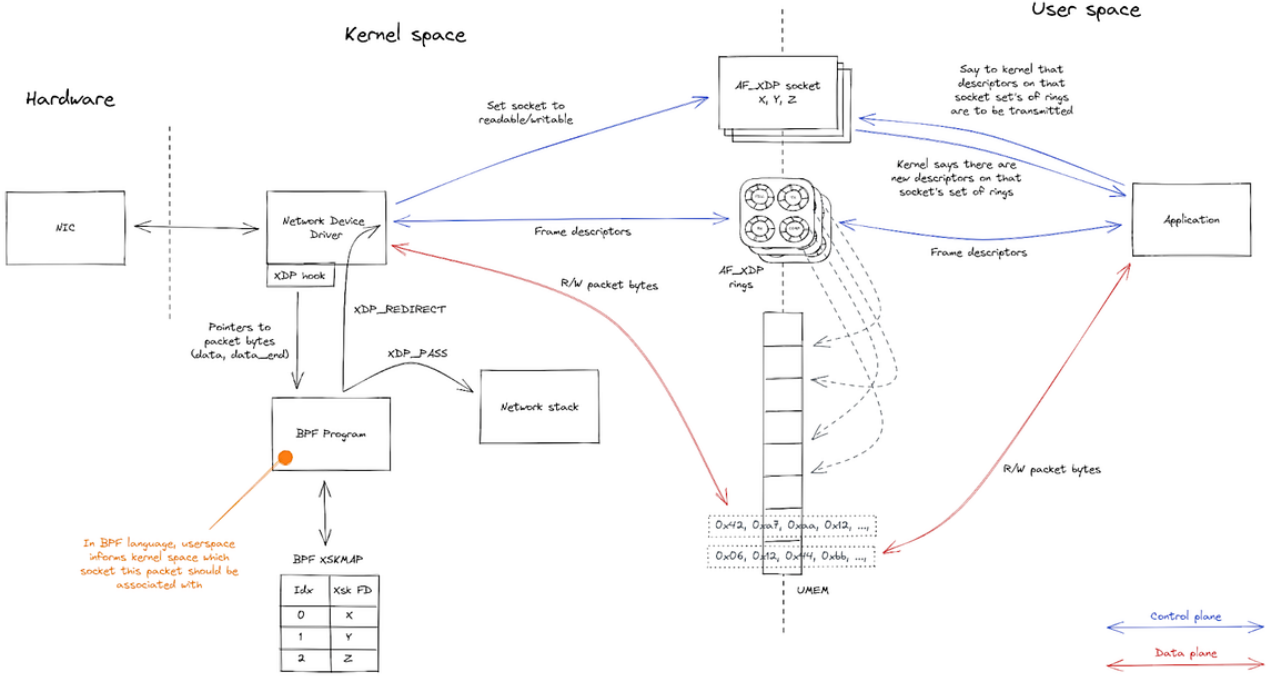
2.4. How Dockerization Helped

The motivation for this Docker-based setup (beyond compliance with project guidelines) was to overcome hardware limitations. The physical network interfaces on our host machines did not support XDP in native mode, instead defaulting to the less performant SKB mode (generic XDP). A significant advantage of the containerized approach was that the virtual `eth0` interface within the Docker container did allow our XDP program to run in native mode. This was required for enabling and testing the zero-copy performance of our AF-XDP implementation. Refer to Fig. 4 and Fig. 5 in the Results section.

3. Implementation details

3.1. The XDP Hook

The `./main/af_xdp_kern.c` file is an XDP program for the hook that filters and redirects incoming UDP packets destined for port 53 (DNS). After verifying that the packet is IPv4 and UDP, it extracts the UDP header and prints the

Fig. 3. AF_XDP Diagram³⁾

source and destination ports using `bpf_printk`. If the destination port is 53, the packet is redirected to a userspace socket via the `xsk_map` (an AF_XDP socket map), enabling fast packet handling without going through the kernel network stack. All other packets are passed up the normal networking path with `XDP_PASS`.

3.2. The XDP Client program

For the client program, we extensively used the `libbpf` library. The above hook program is loaded and attached by the client `af_xdp_client.c`. First loads the hook file using the `xdp_program_create` function and attaches it by calling the `xdp_program_attach` function. After attaching the xdp program, we get the `xsk_map` using the `bpf_object_find_map_by_name` (`((xdp_program_bpf_obj(prog), "xsk_map")` function.

After that we first remove limits on locked memory (`setrlimit`) to ensure enough memory is available for zero-copy packet buffers. Then it allocates a page-aligned memory region (`packet_buffer`) to hold network frames and initializes it as UMEM, a shared memory region between user space and the kernel (`configure_xsk_umem`). Next, it creates and configures an AF_XDP socket (`xsk_configure_socket`) that binds to a network interface queue for high-performance packet I/O. A DNS thread pool is then initialized to handle incoming DNS requests concurrently. The main loop (`rx_and_process`) continuously receives and processes packets, after which the program gracefully cleans up all resources - destroying the thread pool, closing the socket, freeing memory, and exiting successfully.

The function `rx_and_process` continuously polls the XDP socket file descriptor, obtained via `xsk_socket_fd`, using the `poll` system call when `xsk_poll_mode` is enabled. Once packets are available, it calls

`handle_receive_packets()`, which retrieves them from the RX ring buffer through the `xsk_ring_cons_peek` function. To ensure uninterrupted operation, the code refills the UMEM fill queue by checking available frame space using `xsk_prod_nb_free` and reserving entries with `xsk_ring_prod_reserve`. Each frame address is obtained using `xsk_ring_prod_fill_addr` and committed with `xsk_ring_prod_submit`. The function then iterates over all received packets, extracting their memory addresses and lengths from the RX ring and passing them to `process_packet` for deeper inspection and handling.

Each worker thread, implemented by `dns_worker_thread`, runs concurrently and dequeues tasks from the shared queue using `work_queue_pop`. It calls the resolver function `resolve_dns` to perform domain resolution, after which it constructs an appropriate DNS response packet using either `craft_response_with_ip` (for successful resolutions) or `craft_error_response` (for NXDOMAIN or SERVFAIL cases). Responses are transmitted back through the AF_XDP socket using `send_dns_response`, followed by `complete_tx` to notify the kernel that transmission is complete. Finally, each thread releases the UMEM frame with `xsk_free_umem_frame` and frees any allocated memory using `free`. This combination of low-level XDP socket operations, efficient memory management through UMEM, and multithreaded asynchronous DNS resolution enables high-throughput, zero-copy packet handling suitable for ultra-low-latency DNS servers.

4. Custom DNS Resolver

We implement our own custom DNS resolver in the user-space application rather than using a stub resolver. We imple-

ment both a recursive mode and an iterative mode of resolution. We describe the entire workflow of the algorithm used for the iterative mode of resolution as follows:

```

Function DNS_resolve_main(domain_name):
  for IP in ROOT_SERVER_IPs do
    Result
    ← DNS_resolve(domain_name, IP, 0)
    if Result ≠ FAIL then
      | return Result
    end
  end
  return FAIL

Function DNS_resolve(domain_name, ip_address,
depth):
  if depth > 10 then
    | return FAIL
  end
  Response ← query(ip_address, domain_name)
  if Response contains A record then
    | return resolved_IP
  end
  if Response contains CNAME record then
    | return DNS_resolve_main(cname)
  end
  if Response contains NS record then
    if Glue record present then
      | return DNS_resolve(domain_name,
        ns_ip_addresses, depth + 1)
    end
    else
      ns_ip ←
        DNS_resolve_main(ns_domain_name)
      return DNS_resolve(domain_name,
        ns_ip, depth + 1)
    end
  end
  return FAIL

```

Algorithm 1: Iterative DNS Resolution

We further implement caching in our DNS server. We primarily use two caching policies described as follows:

- **HashCache** : This is a very basic and naive algorithm that uses an hashing algorithm to calculate the index of the incoming entry in our cache. If a entry was previously present at that position, it is evicted and replaced with our new entry. Essentially it is random eviction and replacement.
- **Least-Recently-Used Cache**: Once the cache is full, we evict the least recently used (where usage is described as an access to that particular element or its insertion into the cache).

5. Benchmarking setup

We test our implementation on 3 different parameters - latency, throughput and the cpu overhead incurred while performing the DNS resolution.

We mainly use `dnstperf`, a DNS performance benchmarking tool to perform these benchmarks. The scenario we use is as follows:

- **Number of Clients:** **3 Clients** sending DNS queries simultaneously.
- **Maximum number of in-flight (unresolved) queries :** **10**
- **Timeout for each in-flight query :** **15 seconds**.
- **Runtime for the test:** **60 seconds**.

In the host we run the following command while the server is running: `dnstperf -s container_ip -d tmp.txt -c 3 -l 60 -q 10 -t 15 -v`

Here, `tmp.txt` contains 100 domain names for resolution. For systems using windows or some Linux distributions directly using the IP address of the docker container does not work. In such cases replace `container_ip` with `127.0.0.0 -p P`, where P is the port on which the docker container is running.

For benchmarking the CPU overhead in both cases (using AF_XDP sockets versus not), we use the `perf` tool in the `linux-tools` package. This tool is able to measure the number of cycles taken by the CPU for a specified process (specified through the PID). For all the tests we pin the docker container to only **one CPU core**.

6. Results

6.1. Verification of Setup

6.1.1. Without Docker

We connect both the phone and the laptop (running the XDP program on `wlo1` interface) to the same Wi-Fi network (IITGN-SSO). Next, we launch the XDP program on the laptop and use the `dig` command from the phone (via Termux) to query the IP address of the laptop's `wlo1` interface. See Fig. 4.

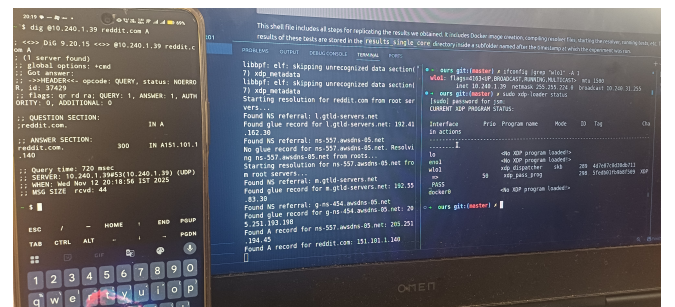


Fig. 4. Without Docker, dig from phone

Notice that the XDP mode is SKB. This issue was described in the Setup section, where, none of our interfaces support XDP in native mode. Hence, we used Docker.

6.1.2. With Docker

We build the Docker container, launch the XDP program, and perform a similar task, this time we dig to the Docker container's IP address. See Fig. 5.

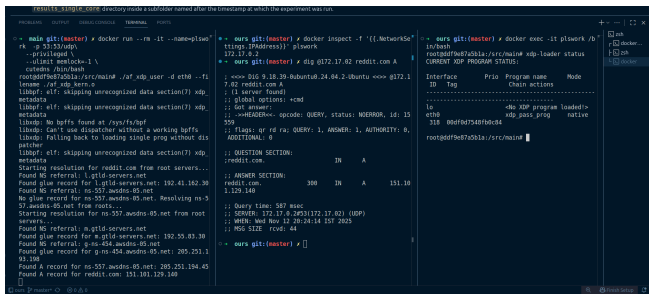


Fig. 5. With Docker, dig to the container's IP address.

Here, we can clearly see that XDP is running in native mode, Hence, no `sk_buff` will be created.

6.2. Latency and throughput

We summarise the results for latency in Table 1.

Table 1. Average and Minimum Latency

Configuration	Average (s)	Min (s)
AFXDP-LRU	0.758775	0.000024
Normal-LRU	1.36117	0.000224
AFXDP-Hashcache	0.916267	0.000028
Normal-Hashcache	1.540820	0.000182

Table 2. Maximum Latency and Standard Deviation

Configuration	Max (s)	Std Dev (s)
AFXDP-LRU	5.191407	1.072431
Normal-LRU	12.421937	2.215635
AFXDP-Hashcache	13.178317	1.639238
Normal-Hashcache	13.298222	2.807204

We observe from Table 1 that the AF_XDP counterparts perform better than their Non-XDP counterparts in terms of average latency. AF_XDP running with the LRU (Least-Recently-Used) Caching policy in the resolver achieves the lowest average latency, achieving an almost 100% decrease in average latency.

Similarly, even when observing standard deviation, we observe lower values for XDP versions than their non XDP counterparts. The lowest value is **1.07** seconds, achieved by AF_XDP running with the LRU caching policy in the resolver. The XDP-Hashcache version is also very close to the above result. Both the XDP version outperform the others by a large margin as observed in Table 2.

We now analyse the results for resolution fraction and throughput in terms of percentage of queries resolved and throughput.

Table 3. Query Statistics

Configuration	Sent	Completed	Lost
AFXDP-LRU	194	156	38
Normal-LRU	214	189	25
AFXDP-Hashcache	196	159	37
Normal-Hashcache	214	190	24

We observe an anomalous behaviour from Table 4. It seems that even though the latency is lowered, the throughput and resolution percentage has also dropped. This anomaly results because of our implementation. We use just **one**

Table 4. Performance Metrics

Configuration	Completion (%)	QPS
AFXDP-LRU	80.41	2.450
Normal-LRU	88.32	2.944
AFXDP-Hashcache	81.12	2.463
Normal-Hashcache	88.79	3.040

XDP socket for all our egress communication. In contrast to this the normal variants spawn a new socket for egress communication i.e sending back the response to the clients and each thread has to acquire a lock to transmit back a response through this. This leads to a bottleneck in our system that leads to losses and timeouts, thus reducing the throughput and resolution percentage. A future prospect would be to spawn multiple sockets for egress too which would improve the throughput metric too. On the bright side, we could argue that the drop in throughput even in the current implementation is not extremely significant.

Finally, we explore the impact of kernel bypass on the CPU overhead. Table 5 demonstrates these results. These results indicate that the XDP variants take upto an order of 3 higher number of cycles per query than their non XDP counterparts. But on the other side, they have much higher Instructions per cycle than their normal counterparts. This indicates that the XDP pathway is much more efficient as it is not riddled with indirection, additional operations and processing that plague the normal linux network pathway.

Table 5. CPU Cycles and Instructions per Query

Configuration	Cycles per Query	Inst. per Cycle
Normal-LRU	1518779	0.26
Normal-Hashcache	1918129	0.23
AFXDP-LRU	1829269090	1.98
AFXDP-Hashcache	1783024441	1.99

References

- 1) BARDINELLI, J., ZHANG, Y., SU, J., HUANG, L., PARILLA, A., JARVI, R., KULKARNI, S. G., AND ZHANG, W. hydns: Acceleration of dns through kernel space resolution. In *Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions* (Aug. 2024), ACM SIGCOMM ’24, ACM, p. 58–64.
- 2) CAI, Q., CHAUDHARY, S., VUPPALAPATI, M., HWANG, J., AND AGARWAL, R. Understanding host network stack overheads. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Aug. 2021), SIGCOMM ’21, ACM, p. 65–77.
- 3) GÄRTNER, M. Recapitulating af_xdp, Mar 2022.
- 4) HØILAND-JØRGENSEN, T., BROUER, J. D., BORKMANN, D., FASTABEND, J., HERBERT, T., AHERN, D., AND MILLER, D. The express data path: fast programmable packet processing in the operating system kernel. In *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies* (New York, NY, USA, 2018), CoNEXT ’18, Association for Computing Machinery, p. 54–66.
- 5) STEPHAN, A., WÜSTRICH, L., OF NETWORK ARCHITECTURES, C., SERVICES, SCHOOL OF COMPUTATION, I., AND TECHNOLOGY, TECHNICAL UNIVERSITY OF MUNICH, G. The path of a packet through the Linux kernel. *Seminar IITM WS 23 89* (2024).
- 6) VIEIRA, M. A. M., CASTANHO, M. S., PACÍFICO, R. D. G., SANTOS, E. R. S., JÚNIOR, E. P. M. C., AND VIEIRA, L. F. M. Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications. *ACM Computing Surveys* 53, 1 (Feb. 2020), 1–36.